

PROJETO INTEGRADOR

Plataforma de Gestão de Eventos

Desafio de Projeto Orientado a Objetos aplicado a uma Plataforma de Gestão de Eventos

Finalidade. Desenvolver a capacidade de analisar um domínio, distribuir responsabilidades entre objetos e construir, em Java, uma solução extensível e testável. A plataforma de eventos é o contexto concreto no qual as decisões de POO serão projetadas, implementadas, demonstradas e avaliadas.

Campo	Definição
Disciplina	Programação Orientada a Objetos II
Modalidade	Projeto em equipes de 3 a 4 estudantes
Produto	Uma implementação independente por equipe, baseada nos mesmos requisitos
Prazo final	Primeira semana de outubro de 2026
Ritmo	Quinta-feira: orientação; terça-feira: demonstração do incremento
Versão	1.1 — revisão com centralidade em POO — 10 de agosto de 2026

Documento-base para apresentação do desafio, planejamento das equipes, acompanhamento semanal e avaliação do produto.

Sumário executivo do documento

1. Termo de abertura e visão do produto
2. Escopo, prioridades e limites da primeira versão
3. Atores e responsabilidades
4. Requisitos funcionais
5. Regras de negócio
6. Requisitos não funcionais e restrições
7. Cenários ponta a ponta e critérios de aceitação
8. Diretrizes de arquitetura e decisões tecnológicas
9. Processo de desenvolvimento e entregas semanais
10. Avaliação da disciplina e definição de pronto
11. Artefatos finais e roteiro de demonstração

Leitura recomendada. As equipes devem começar pelas seções 1 a 3, identificar os requisitos obrigatórios na seção 4 e planejar um fluxo ponta a ponta executável desde as primeiras semanas. A modelagem de classes não é fornecida: ela integra o trabalho de projeto a ser avaliado.

Convenções utilizadas

Classificação	Significado
Obrigatório (M)	Deve estar implementado e demonstrável, ainda que por uma solução simplificada.
Desejável (D)	Amplia o valor do produto e pode ser implementado depois do núcleo obrigatório.
Opcional / plus (P)	Desafio de extensão; não substitui requisitos obrigatórios pendentes.

1. Termo de abertura e visão do produto

1.1 Contexto e problema

Eventos acadêmicos e profissionais frequentemente utilizam ferramentas desconectadas para divulgação, inscrições, programação, controle de frequência, avaliações e certificados. Essa fragmentação aumenta o trabalho manual, dificulta a consolidação de dados e limita a experiência do participante. O desafio é projetar uma plataforma configurável, inicialmente aplicada a um simpósio acadêmico, mas sem regras fixas que a restrinjam a um único tipo de evento.

1.2 Objetivo do produto

Construir uma plataforma de gestão de eventos capaz de cadastrar eventos e atividades, publicar uma página web, receber inscrições, organizar a agenda pessoal do participante, registrar frequência por QR Code com

alternativa manual, coletar avaliações e emitir relatórios. A aplicação desktop e o site público deverão compartilhar a mesma API e o mesmo banco de dados.

1.3 Objetivos de aprendizagem

1. Modelar o domínio e distribuir comportamento entre classes e objetos com responsabilidades claras, alta coesão e baixo acoplamento.
2. Proteger invariantes por encapsulamento e aplicar composição, polimorfismo e interfaces nos pontos reais de variação do domínio.
3. Usar padrões de projeto para resolver problemas reais, sem aplicação ornamental.
4. Separar domínio, casos de uso e infraestrutura por portas e adaptadores, mantendo as regras centrais independentes de telas e banco.
5. Evoluir o modelo por testes e refatorações, entregando semanalmente incrementos executáveis e integrados.
6. Justificar decisões técnicas e comunicar a evolução do produto com evidências.

1.4 Desafio de Projeto Orientado a Objetos

O sistema de eventos não é o fim pedagógico do trabalho. Ele funciona como um domínio suficientemente rico para que a equipe demonstre capacidade de projetar uma solução orientada a objetos. Telas, endpoints e tabelas são necessários, mas não substituem um modelo que concentre comportamento, proteja regras e suporte variações sem condicionais espalhadas.

A equipe deverá demonstrar, no código e nas explicações, como as responsabilidades foram atribuídas, quais invariantes são protegidas pelos objetos, onde existe polimorfismo, por que uma interface ou padrão foi criado e como testes e refatorações melhoraram o projeto.

Critério central. Uma solução visualmente completa, mas procedural, anêmica ou excessivamente concentrada em controladores e serviços, não demonstra o objetivo principal de POO II. A funcionalidade será avaliada em conjunto com a qualidade do modelo que a sustenta.

1.5 Partes interessadas

Parte	Interesse e responsabilidade
Professor	Cliente pedagógico e avaliador; esclarece o problema, valida incrementos e avalia decisões.
Equipe de estudantes	Responsável por análise, modelagem, implementação, testes, documentação e demonstração.
Organização do evento	Configura eventos, atividades, inscrições, presença, avaliações e relatórios.
Participante	Consulta programação, inscreve-se, monta agenda, registra presença e avalia atividades elegíveis.
Público visitante	Consulta informações públicas e inicia inscrição pelo site.

2. Escopo, prioridades e limites

2.1 Núcleo obrigatório da primeira versão

1. Cadastro e autenticação por e-mail e senha, sem recuperação de senha nesta versão.
2. Gestão de eventos, atividades, categorias/trilhas, locais, horários e pessoas vinculadas.
3. Site público simples, alimentado pela mesma API e pelo mesmo banco de dados.
4. Inscrição configurável no evento e em atividades, com agenda personalizada.
5. Controle de frequência configurável por atividade, por QR Code e lançamento manual.
6. Avaliação configurável de atividades e apresentações por participante inscrito com presença validada.
7. Relatórios operacionais de inscrições, frequência e participação.

2.2 Extensões desejáveis e opcionais

Classe	Extensão
D	Geração de certificado em PDF para participante elegível.
D	Envio de certificados em PDF aos e-mails cadastrados.
D	Certificado ou declaração para palestrantes e apresentadores.
P	Lista de espera e tratamento automático de vagas liberadas.
P	Rede social interna do congresso, com interações moderadas.
P	Recursos avançados de análise, notificações ou personalização.

2.3 Fora do escopo obrigatório

1. Aplicativo móvel nativo.
2. Recuperação de senha, login social e autenticação multifator.
3. Processamento de pagamentos e emissão de nota fiscal.
4. Infraestrutura de produção em alta disponibilidade.
5. Integração obrigatória com Google Drive, serviços institucionais ou equipamentos específicos.

Regra de priorização. Uma extensão não compensa a ausência de requisito obrigatório. Primeiro deve existir um núcleo integrado e demonstrável; depois a equipe poderá investir em funcionalidades desejáveis ou opcionais.

3. Atores e responsabilidades no sistema

Ator	Responsabilidades mínimas
------	---------------------------

Ator	Responsabilidades mínimas
Administrador	Gerencia usuários organizadores, parâmetros gerais e acesso administrativo; pode consultar dados de todos os eventos.
Organizador	Cria e mantém eventos, atividades, programação, inscrições, critérios de frequência, avaliações e relatórios.
Participante	Mantém seu cadastro, inscreve-se, escolhe atividades, consulta agenda, registra presença e avalia quando elegível.
Visitante	Acessa o site público sem autenticação para consultar eventos e iniciar cadastro/inscrição.

Decisão de domínio. “Avaliador” não é um perfil separado. Avaliar é uma capacidade temporária do participante que esteja inscrito e tenha presença validada na atividade correspondente.

4. Requisitos funcionais

Os requisitos a seguir descrevem comportamentos observáveis. A equipe deverá refiná-los em casos de uso, responsabilidades de objetos e testes, preservando seu significado. “Implementação simplificada” reduz sofisticação, mas não elimina o fluxo, a persistência, a regra central nem a evidência de POO. Atender às telas sem uma modelagem coerente não é suficiente.

4.1 Acesso e usuários

ID	Requisito	Prio.	Critério mínimo de aceitação
RF-01	Permitir cadastro de usuário com nome, e-mail e senha.	M	Novo usuário consegue cadastrar-se; e-mail duplicado é rejeitado com mensagem clara.
RF-02	Autenticar usuários e aplicar permissões conforme o perfil.	M	Credenciais válidas iniciam sessão; funções protegidas não ficam disponíveis a perfis sem autorização.
RF-03	Permitir ao participante consultar e atualizar seus dados básicos.	M	Alterações válidas persistem e aparecem na próxima consulta.

4.2 Eventos, atividades e programação

ID	Requisito	Prio.	Critério mínimo de aceitação
RF-04	Criar, editar, consultar e encerrar eventos.	M	Organizador mantém título, descrição, período, local/modalidade, estado e regras gerais.
RF-05	Cadastrar atividades de diferentes tipos, sem restringir o evento a uma categoria acadêmica.	M	É possível representar palestra, apresentação oral, pôster, produto, mesa, oficina ou outro tipo configurado.
RF-06	Organizar atividades por trilhas, categorias ou espaços.	M	Atividades podem ser filtradas e agrupadas na programação.

ID	Requisito	Prio.	Critério mínimo de aceitação
RF-07	Cadastrar horários e locais, detectando conflitos relevantes.	M	O sistema impede ou alerta sobre conflito de local/horário segundo decisão documentada pela equipe.
RF-08	Vincular pessoas às atividades com papéis como palestrante, apresentador ou responsável.	M	A programação identifica as pessoas e seus papéis; fotografia é opcional.
RF-09	Consultar a programação geral por data, horário, trilha, tipo ou local.	M	Usuário encontra atividades por ao menos três filtros combináveis.

4.3 Site público e inscrições

ID	Requisito	Prio.	Critério mínimo de aceitação
RF-10	Publicar página web simples com os eventos disponíveis.	M	Visitante visualiza eventos publicados com informações essenciais.
RF-11	Exibir programação e pessoas em destaque no site.	M	Atividades e palestrantes são exibidos; foto ausente não impede cadastro nem publicação.
RF-12	Permitir cadastro e inscrição a partir do site público.	M	Visitante completa o fluxo até confirmação da inscrição.
RF-13	Configurar a regra de inscrição por evento e por atividade.	M	Organizador define se basta inscrição no evento, se há escolha de atividades e se vagas são controladas.
RF-14	Controlar vagas quando essa regra estiver ativada.	M	O sistema não confirma inscrições acima do limite; comportamento alternativo é informado.
RF-15	Permitir cancelamento de inscrição conforme regra configurada.	M	Cancelamento dentro do prazo altera a situação e libera vaga quando aplicável.

4.4 Agenda do participante

ID	Requisito	Prio.	Critério mínimo de aceitação
RF-16	Permitir ao participante selecionar atividades de interesse.	M	Seleções válidas compõem sua agenda pessoal.
RF-17	Apresentar agenda pessoal em ordem cronológica.	M	A agenda mostra somente atividades escolhidas/inscritas, com data, horário e local.
RF-18	Alertar ou impedir seleção de atividades com horários conflitantes.	M	Conflito é tratado de modo consistente e explicado ao usuário.

4.5 Frequência

ID	Requisito	Prio.	Critério mínimo de aceitação
RF-19	Configurar o critério de presença de cada atividade.	M	Organizador escolhe, ao menos, check-in único, entrada e saída ou validação manual.

ID	Requisito	Prio.	Critério mínimo de aceitação
RF-20	Gerar QR Code associado à operação de frequência.	M	Código válido permite registrar a ação prevista sem expor senha ou dado sensível.
RF-21	Registrar frequência por leitura do QR Code.	M	Registro válido é persistido com participante, atividade, data/hora e tipo de marcação.
RF-22	Permitir lançamento manual alternativo por usuário autorizado.	M	Organizador registra ou corrige presença com identificação de quem realizou a operação.
RF-23	Calcular a situação de presença conforme a regra da atividade.	M	Situação é reproduzível a partir dos registros e da configuração vigente.

4.6 Avaliações

ID	Requisito	Prio.	Critério mínimo de aceitação
RF-24	Criar questionários configuráveis para atividades ou apresentações.	M	Organizador define título, perguntas e tipos de resposta.
RF-25	Suportar ao menos resposta textual, escolha única e escala numérica.	M	Cada tipo é exibido, validado e persistido corretamente.
RF-26	Restringir a avaliação a participante inscrito e com presença validada.	M	Usuário inelegível não envia avaliação; motivo é apresentado.
RF-27	Evitar múltiplas respostas do mesmo participante ao mesmo questionário.	M	Nova tentativa é bloqueada ou atualiza a resposta, conforme regra documentada.
RF-28	Consolidar resultados das avaliações.	M	Organizador consulta quantidades, distribuições simples e comentários, preservando a identificação conforme política adotada.

4.7 Relatórios e certificados

ID	Requisito	Prio.	Critério mínimo de aceitação
RF-29	Emitir relatório de inscritos por evento e atividade.	M	Relatório contém identificação, situação e totalizações básicas.
RF-30	Emitir relatório de frequência e participação.	M	Relatório apresenta registros e situação calculada por participante/atividade.
RF-31	Exportar ao menos um relatório em formato portátil.	M	Um relatório pode ser salvo em CSV ou PDF e aberto fora do sistema.
RF-32	Configurar critério de elegibilidade para certificado por evento.	D	Organizador define percentual mínimo ou conjunto de atividades obrigatórias.
RF-33	Gerar certificado em PDF para participante elegível.	D	Documento contém dados essenciais, identificador verificável e é baixável.
RF-34	Enviar certificado em PDF ao e-mail cadastrado.	D	Envio é registrado; falha não elimina a possibilidade de download.

ID	Requisito	Prio.	Critério mínimo de aceitação
RF-35	Gerar certificado/declaração para palestrante ou apresentador.	D	Documento reflete atividade e papel exercido.

4.8 Extensão social

ID	Requisito	Prio.	Critério mínimo de aceitação
RF-36	Disponibilizar espaço de interação entre participantes do evento.	P	A equipe define um recorte seguro e moderável, como mural de mensagens ou conexões.

5. Regras de negócio

ID	Regra
RN-01	Um e-mail identifica uma única conta de usuário.
RN-02	A fotografia de uma pessoa é opcional; sua ausência não bloqueia cadastro ou publicação.
RN-03	Um evento pode possuir várias atividades simultâneas, trilhas, tipos e locais.
RN-04	O estado do evento controla sua visibilidade e a abertura de inscrições.
RN-05	A inscrição pode ser configurada no nível do evento e complementada por regras de atividade.
RN-06	A escolha de atividade não pode exceder vagas quando o controle estiver ativado.
RN-07	Atividades conflitantes devem ser alertadas ou bloqueadas por uma política uniforme e documentada.
RN-08	A agenda pessoal deriva das escolhas e inscrições válidas do participante.
RN-09	Cada atividade possui sua própria política de frequência.
RN-10	O QR Code não deve conter senha nem dado pessoal sensível em texto aberto.
RN-11	Registros de frequência devem guardar data/hora, origem e responsável quando manuais.
RN-12	Correções manuais de frequência devem preservar rastreabilidade mínima.
RN-13	Somente participante inscrito e com presença validada pode avaliar a atividade.
RN-14	Cada participante possui no máximo uma resposta válida por questionário, salvo edição explicitamente permitida.
RN-15	A política de identificação ou anonimato das respostas deve ser informada antes do envio.
RN-16	A elegibilidade para certificado, quando implementada, decorre de critério configurado e calculável.
RN-17	Mudanças de regra não podem alterar silenciosamente registros históricos já consolidados.
RN-18	Somente administrador ou organizador autorizado altera dados operacionais do evento.
RN-19	O mesmo banco de dados é a fonte compartilhada pela aplicação desktop, API e site.
RN-20	Horários e datas devem considerar o fuso definido para o evento.

6. Requisitos não funcionais e restrições

ID	Qualidade	Exigência
RNF-01	Arquitetura	Separar domínio/aplicação de interface, persistência e integrações; arquitetura hexagonal é a referência recomendada.
RNF-02	Integração	Desktop e site devem consumir a mesma API e compartilhar o mesmo banco de dados.

ID	Qualidade	Exigência
RNF-03	Tecnologia	Java é obrigatório no núcleo avaliado de POO; frameworks, interface, banco e tecnologias web são escolhidos e justificados pela equipe.
RNF-04	Persistência	Usar banco de dados relacional com criação reproduzível do esquema e dados mínimos de demonstração.
RNF-05	Segurança	Senhas devem ser armazenadas por hash apropriado; nunca em texto puro.
RNF-06	Autorização	Operações protegidas devem ser verificadas no servidor/API, e não apenas ocultadas na interface.
RNF-07	Validação	Entradas inválidas devem ser rejeitadas com mensagens compreensíveis e sem corromper dados.
RNF-08	Privacidade	Coletar apenas dados necessários e informar a finalidade; dados de avaliação devem seguir a política declarada.
RNF-09	Usabilidade	Fluxos essenciais devem ser concluídos sem conhecimento técnico e com feedback visível.
RNF-10	Desempenho	Consultas comuns devem responder de forma adequada em uma base de demonstração com, no mínimo, 500 participantes e 100 atividades.
RNF-11	Confiabilidade	Operações críticas não podem produzir duplicidade por repetição acidental.
RNF-12	Testabilidade	Regras centrais devem possuir testes automatizados independentes da interface.
RNF-13	Manutenibilidade	Código deve apresentar nomes expressivos, responsabilidades delimitadas e dependências justificáveis.
RNF-14	Observabilidade	Falhas relevantes devem ser registradas sem expor senhas ou dados sensíveis.
RNF-15	Portabilidade	A equipe deve fornecer instruções suficientes para executar o projeto em outro computador.

6.1 Liberdade tecnológica com responsabilidade

Cada equipe escolherá as tecnologias e deverá justificar como elas atendem ao problema e aos objetivos da disciplina. A justificativa deve comparar ao menos uma alternativa e explicar impactos em modelagem, integração, testes e execução. A liberdade não autoriza substituir o núcleo orientado a objetos em Java por uma solução pronta ou predominantemente configurada por ferramentas de baixo código.

7. Cenários ponta a ponta e critérios de aceitação

Cenário	Demonstração esperada
CA-01 — Publicação	Organizador cria evento, cadastra atividades e pessoas, publica e visualiza os mesmos dados no site.
CA-02 — Inscrição	Visitante cadastra conta, inscreve-se no evento e recebe confirmação consultável.
CA-03 — Agenda	Participante escolhe atividades, recebe tratamento de conflito e consulta agenda cronológica.
CA-04 — Frequência QR	Participante elegível registra presença por QR Code e o sistema calcula a situação conforme

Cenário	Demonstração esperada
	a regra da atividade.
CA-05 — Alternativa manual	Organizador registra/corriga presença manualmente e o sistema preserva autoria e horário da operação.
CA-06 — Avaliação	Participante inscrito e presente responde questionário; outro usuário inelegível é impedido.
CA-07 — Relatório	Organizador consulta e exporta relatório coerente com inscrições e presenças registradas.
CA-08 — Certificado (desejável)	Participante elegível obtém PDF; participante inelegível visualiza a justificativa.

Aceitação do núcleo. Na entrega final, os cenários CA-01 a CA-07 devem funcionar de forma integrada. Dados inseridos no desktop devem aparecer no site e vice-versa quando o fluxo permitir. Não serão aceitas telas isoladas, protótipos sem persistência ou demonstrações baseadas apenas em slides.

8. Diretrizes de arquitetura e decisões técnicas

8.1 Estrutura lógica esperada

1. **Domínio:** objetos, regras e invariantes do negócio, sem dependência direta de interface ou banco.
2. **Aplicação:** casos de uso que coordenam o domínio e definem portas de entrada e saída.
3. **Adaptadores de entrada:** interface desktop e API REST.
4. **Adaptadores de saída:** persistência, e-mail, geração de PDF e demais integrações.
5. **Site público:** cliente da API compartilhada, sem acesso direto ao banco de dados.

8.2 Decisões que a equipe deve registrar

ID	Decisão arquitetural
D-01	Framework de interface desktop e motivo da escolha.
D-02	Framework da API e estratégia de autenticação/autorização.
D-03	Banco relacional, mapeamento e migração do esquema.
D-04	Tecnologia do site público e forma de consumo da API.
D-05	Estratégia de QR Code, validade e prevenção de duplicidade.
D-06	Modelagem das políticas configuráveis de inscrição, frequência, avaliação e certificado, com alternativas consideradas.
D-07	Padrões de projeto usados, problema resolvido, efeito no acoplamento e alternativa descartada.
D-08	Estratégia de testes, refatorações do modelo e preparação dos dados de demonstração.

8.3 Evidências mínimas de orientação a objetos

1. Modelo de domínio coerente com o vocabulário dos requisitos.
2. Invariantes protegidas nos objetos apropriados, não apenas na interface.
3. Uso preferencial de composição; herança somente quando representar uma relação justificável.
4. Interfaces em pontos de variação ou integração, evitando abstrações sem necessidade.
5. Ao menos dois padrões de projeto aplicados e explicados por seu efeito no código.
6. Testes de regras que demonstrem comportamento e não apenas cobertura superficial.

8.4 Requisitos técnicos avaliáveis de POO

Os itens ROO são obrigatórios e integram a definição de pronto do projeto. Eles não prescrevem uma lista artificial de recursos da linguagem; exigem evidências de decisões adequadas ao domínio.

ID	Aspecto	Evidência mínima
ROO-01	Modelo de domínio	Representar conceitos do problema com vocabulário consistente e responsabilidades explícitas; evitar classes que sejam apenas recipientes de dados.
ROO-02	Encapsulamento	Proteger invariantes e mudanças de estado dentro dos objetos responsáveis, sem setters públicos indiscriminados.
ROO-03	Objetos de valor	Usar objetos de valor e imutabilidade quando identidade não for necessária e a validação puder acompanhar o conceito.
ROO-04	Composição e colaboração	Preferir colaboração entre objetos e composição de comportamentos; justificar dependências e direção das associações.
ROO-05	Polimorfismo	Tratar ao menos um ponto real de variação por comportamento polimórfico, evitando cadeias extensas de if/switch por tipo.
ROO-06	Herança	Usar herança somente quando houver relação conceitual substituível; ausência de herança é aceitável quando composição for melhor.
ROO-07	Interfaces	Definir interfaces em fronteiras externas ou pontos de variação, com contratos pequenos e orientados às necessidades do cliente.
ROO-08	SOLID	Demonstrar coesão, inversão de dependência e extensibilidade; explicar pelo menos uma refatoração motivada por esses princípios.
ROO-09	Arquitetura hexagonal	Manter domínio e aplicação independentes de interface, framework e persistência, conectados por portas e adaptadores.
ROO-10	Padrões de projeto	Aplicar e justificar ao menos dois padrões que resolvam problemas concretos; nomear padrão sem efeito no projeto não conta como evidência.
ROO-11	Erros do domínio	Modelar falhas e estados inválidos de forma explícita, preservando consistência e mensagens compreensíveis.
ROO-12	Testes e refatoração	Testar comportamentos centrais sem depender da interface e registrar ao menos duas refatorações relevantes do modelo.

8.5 Problemas do domínio que favorecem POO

Variação	Exemplos	Possibilidade de projeto
Validação de frequência	Check-in, entrada/saída, percentual de permanência ou conferência manual.	Strategy ou composição de políticas; regra selecionada pela configuração da atividade.
Inscrição	Vagas livres, lista de espera, aprovação ou atividade sem controle de capacidade.	Políticas intercambiáveis e objetos que preservam capacidade e estado da inscrição.
Certificação	Percentual de frequência, atividades obrigatórias ou liberação do organizador.	Specification/Strategy e composição de critérios de elegibilidade.
Questionários	Perguntas de escala, escolha, texto e respostas obrigatórias ou opcionais.	Composição ou hierarquia de perguntas e validação polimórfica de respostas.
Publicação e integrações	Banco, site, QR Code, e-mail e PDF.	Ports and Adapters; dependências externas substituíveis por adaptadores.
Tipos de atividade	Palestra, artigo, pôster, oficina e outras categorias configuráveis.	Composição de características; evitar subclasses sem comportamento ou condicionais por tipo.

Nota: **as possibilidades acima são referências de raciocínio, não soluções obrigatórias**. Cada equipe deverá comparar alternativas e justificar a escolha que produza o modelo mais simples e extensível para seu recorte.

9. Processo de desenvolvimento e entregas semanais

Cada equipe produzirá sua própria versão. O trabalho será acompanhado em ciclos semanais. Quintas-feiras são destinadas a orientação, decisões e remoção de impedimentos. Terças-feiras são destinadas à demonstração de software funcionando. O incremento apresentado deve estar integrado ao repositório principal da equipe.

Sem.	Encontros	Foco	Incremento demonstrável
1	11 e 13/08	Domínio, responsabilidades e backlog	Repositório e aplicação iniciando; glossário do domínio, mapa de responsabilidades e primeiro modelo justificado.
2	18 e 20/08	Objetos, invariantes e persistência	Evento e atividade persistidos; entidades/objetos de valor protegendo regras; testes iniciais.
3	25 e 27/08	Casos de uso, portas e autenticação	Cadastro/login e autorização; interfaces de entrada/saída; dependências invertidas e teste sem interface.
4	01 e 03/09	Adaptadores, site público e inscrição	Site e desktop consumindo a API; inscrição integrada; separação entre DTOs, aplicação e domínio.
5	08 e 10/09	Políticas de inscrição e agenda	Agenda e conflitos; ao menos um ponto de variação resolvido por composição/polimorfismo.
6	15 e 17/09	Frequência configurável	QR Code e alternativa manual; estratégias de frequência, invariantes, rastreabilidade e testes.
7	22 e 24/09	Avaliações, relatórios e refatoração	Elegibilidade e questionário configurável; padrão justificado; comparação antes/depois de uma refatoração.
8	29/09 e 01/10	Integração, qualidade e entrega final	Cenários completos; ROO-01 a ROO-12 rastreados; testes, documentação e demonstração final.

9.1 Protocolo da demonstração de quinta-feira

1. Indicar a meta assumida na semana e o requisito relacionado.
2. Executar o incremento em ambiente preparado pela equipe.
3. Mostrar ao menos um caminho válido e um tratamento de erro.
4. Apresentar um teste relevante e a alteração no modelo ou arquitetura.
5. Registrar pendências, decisão tomada e próxima meta.
6. Apresentar uma decisão de POO da semana: responsabilidade movida, invariante protegida, abstração criada/removida ou acoplamento reduzido.
7. Comparar, quando houver refatoração, o problema do código anterior com o efeito observável da nova solução.

Evidência de progresso. Código não integrado, telas estáticas, trechos isolados e apresentações sem execução não caracterizam incremento concluído. Uma entrega pequena, integrada e testada vale mais como evidência do que muitas funcionalidades parcialmente conectadas.

10. Avaliação da disciplina e definição de pronto

Dimensão	Peso	Evidência principal
Modelagem orientada a objetos	30%	Responsabilidades, comportamento no domínio, encapsulamento de invariantes, coesão, acoplamento, composição e polimorfismo.
Arquitetura e qualidade do código	20%	Portas e adaptadores, SOLID, dependências, legibilidade, tratamento de erros e padrões justificados.
Testes e evolução do projeto	10%	Testes de comportamento, refatorações demonstradas e confiabilidade das regras centrais.
Atendimento funcional e integração	20%	Requisitos obrigatórios, API e banco compartilhados, persistência e cenários ponta a ponta executáveis.
Processo incremental e participação	15%	Entregas semanais, evolução observável, gestão de escopo e contribuição dos integrantes.
Documentação e comunicação	5%	README, decisões, rastreabilidade entre RFs e ROOs e clareza da demonstração final.

Regra de interpretação da nota

Os três primeiros critérios somam 60% e representam o núcleo da disciplina. Interface sofisticada, grande quantidade de funcionalidades ou uso de frameworks não compensam um projeto procedural, um domínio anêmico ou decisões que a equipe não consegue explicar. **Não se exige complexidade artificial: abstrações e padrões só recebem crédito quando resolvem um problema demonstrável.**

10.1 Condições de avaliação do escopo

1. Todos os requisitos classificados como obrigatórios devem estar presentes, ainda que com interface ou automação simplificada.
2. A simplificação deve ser declarada e não pode eliminar regra central, persistência, integração ou autorização.
3. Funcionalidades desejáveis e opcionais só agregam valor depois de estabilizado o núcleo obrigatório.
4. A avaliação considera o código e a execução; não apenas a aparência das telas.
5. A equipe deve conseguir explicar as próprias decisões, inclusive trechos produzidos com apoio de IA ou bibliotecas.

10.2 Definição de pronto para cada requisito

Condição	Evidência
Implementado	O comportamento está disponível no fluxo correspondente.
Integrado	Usa API, banco e regras reais; não é uma simulação isolada.
Validado	Caminho principal e ao menos um erro relevante foram verificados.

Condição	Evidência
Testado	Há teste automatizado quando o requisito envolve regra de domínio.
Documentado	README/backlog indicam como executar e qual decisão foi tomada.
Demonstrável	Pode ser executado diante do professor com dados controlados.
Sustentado por POO	A responsabilidade está no objeto adequado, invariantes são protegidas e variações não dependem de condicionais espalhadas.

11. Artefatos finais e roteiro de demonstração

11.1 Entregáveis obrigatórios

1. Código-fonte versionado e histórico de contribuições da equipe.
2. README com pré-requisitos, configuração, execução e usuários de demonstração.
3. Modelo de domínio, mapa de responsabilidades e diagrama arquitetural atualizados, coerentes com o código final.
4. Registro curto das decisões D-01 a D-08 e das principais refatorações, com problema, alternativa e efeito no projeto.
5. Script de criação/migração do banco e conjunto de dados para demonstração.
6. Coleção ou documentação dos endpoints da API.
7. Testes automatizados e instruções para executá-los.
8. Matriz relacionando RFs e ROOs às responsabilidades, à implementação e ao teste/demonstração correspondente.

11.2 Roteiro sugerido para a apresentação final

1. Apresentar em até dois minutos o problema, o recorte e as escolhas tecnológicas.
2. Demonstrar os cenários CA-01 a CA-07 com uma base preparada.
3. Explicar duas decisões relevantes de modelagem, dois padrões de projeto e uma refatoração com evidência de melhoria.
4. Executar testes selecionados e mostrar tratamento de uma falha.
5. Encerrar com limitações conhecidas, extensão implementada e próximos passos.

11.3 Matriz de rastreabilidade a manter pela equipe

Requisito	ROO relacionado	Responsabilidade implementação /	Teste ou demonstração	Status
RF-__	ROO-__	Classe(s) / caso de uso	Evidência	Situação
RF-__	ROO-__	Classe(s) / caso de uso	Evidência	Situação
RF-__	ROO-__	Classe(s) / caso de uso	Evidência	Situação

Encerramento

O produto é o laboratório no qual a **equipe demonstra projeto orientado a objetos**. A qualidade do trabalho será evidenciada pela capacidade de transformar requisitos configuráveis em objetos que colaboram, protegem invariantes e admitem evolução, sustentados por arquitetura, testes, refatoração e incrementos executáveis.

Pergunta orientadora: como a POO sustenta eventos diversos?